1. Extracting a Musical Foreground from Stereo Ambience

ELEC5305 Acoustics, Speech and Signal Processing | Audio Processing Report 2

Author: Chenyue Wang | Student ID: 540262760

Date: 30 October 2026

2. Objective

This report investigates whether spatial similarity between two audio channels can recover a musical foreground from background noise. Building on Week 8 stereo decomposition and Week 6 overlap-add processing, it compares a covariance-based Wiener estimator with ordinary channel averaging. Controlled experiments vary source panning, background correlation and covariance smoothing time. The objective is to quantify both reconstruction accuracy and preservation of stereo level differences, and to identify a situation where a coherent foreground model fails.

3. Background

A stereo mixture can be modelled as x = g*s + n, where s is a single foreground source, g contains its channel gains and n is background sound. Week 8 uses interchannel level difference (ICLD), interchannel correlation (ICC), and linear prediction weights to separate direct and ambient components [1]. Spatial decomposition also motivates alternative playback arrangements [2]. Correlation indicates shared structure, not whether a sound is desirable: a common noise source can resemble the foreground.

For equal-power, uncorrelated background channels, the covariance is R = P_s*g*g' + P_n*I. Its smaller eigenvalue estimates background power. Its principal eigenvector estimates the foreground direction. A Wiener estimator attenuates that component according to its signal-to-total-power ratio. A short-time Fourier transform (STFT) applies this model separately in frequency bins. Windowed inverse transforms are combined by overlap-add [3,4]. Longer smoothing stabilises covariance estimates but reacts more slowly to changes. This implementation assumes amplitude panning without interchannel delay and does not implement a full room or hearing model.

4. Methodology and Implementation

Figure 1 outlines the processing sequence. make_scene.m synthesises six notes at 16 kHz using fundamentals 330, 440, 392, 523, 440 and 659 Hz, with second and third harmonics and smooth 0.65 s envelopes. Notes start every 0.8 s after 0.5 s of background. Independent Gaussian streams pass through 1/(1-0.75*z^-1); a shared stream controls background correlation rho. Three scenes use (right/left gain, rho) = (1,0), (0.4,0), and (1,0.8). Background power is calibrated to exactly 0 dB aggregate stereo input signal-to-noise ratio (SNR) over 0.5 <= t < 5.3 s, including note gaps.

Each 512-sample frame (32 ms) advances by 128 samples (8 ms). Square-root periodic Hann analysis and synthesis windows use explicit window-square normalisation. fft and ifft perform the transforms [4]. The per-bin real covariance uses exponential memory a = exp(-H/(fs*tau)), with tau equal to 20, 80 or 240 ms. Its eigenvalues lambda_1 >= lambda_2 and unit principal eigenvector v give the following estimator. Small denominators are guarded.

$$\hat{\mathbf{s}} = G\mathbf{v}\mathbf{v}^T\mathbf{x}, \qquad G = \frac{\lambda_1 - \lambda_2}{\lambda_1}.$$

This is a simplified per-bin version of the Week 8 model: it omits cochlear aggregation and post-scaling. Clean references are used only for evaluation. Five fixed noise seeds (5306-5310) are paired across methods; all 45 configurations are reported. A mid-channel baseline sends (L+R)/2 to both outputs. No method receives the true panning gains or noise level.

```
report_figures('pipeline');
```

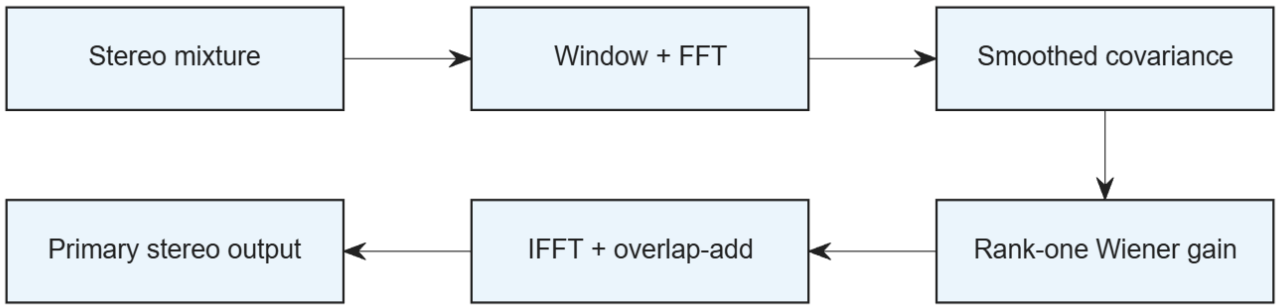


Figure 1. Processing pipeline. Synthesis and scoring are separate from the estimator; no clean signal enters its covariance calculation.

5. Code and Results

Run this Live Script from its submitted folder. The three main helpers separate synthesis, processing and evaluation; report_figures.m contains plotting code. No audio download or additional toolbox is needed. The following call regenerates every experiment, CSV, MAT file and WAV example.

```
clearvars;
close all;
R = run_experiment();
fprintf('Maximum bypass reconstruction error: %.2e\n',R.identityError);
```

```
Maximum bypass reconstruction error: 2.22e-16
```

```
fprintf('Analytic Wiener-matrix check error: %.2e\n',R.wienerError);
```

```
Analytic Wiener-matrix check error: 3.33e-16
```

The bypass check retains every spectrum unchanged and tests padding, window normalisation and trimming. The second check compares eigenvalue weights with a direct Wiener matrix for a known covariance. Assertions additionally check output length, finite results and exact input SNR.

Reconstruction SNR below includes both residual noise and source distortion. The sums cover both channels and the same fixed time region; there is no per-output gain fitting or time alignment.

$$Q = 10 \log_{10} \frac{\sum_{n,c} s_c[n]^2}{\sum_{n,c} (\hat{s}_c[n] - s_c[n])^2}.$$

5.1. Parameter Study

Table 1 reports mean output SNR and sample standard deviation (SD) over the five noise realisations. SD describes this small synthetic study, not uncertainty across real music recordings. Input SNR is 0 dB throughout.

```
fprintf('Table 1. Reconstruction SNR, mean +/- SD (dB)\n');
```

```
Table 1. Reconstruction SNR, mean +/- SD (dB)
```

```
for scene = 1:3
    fprintf('\n%s\n',R.labels{scene});
    sel = R.table.Scene == scene & R.table.Tau_ms == 80;
    fprintf('Mid baseline: %.2f +/- %.2f\n', ...
        mean(R.table.Mid_dB(sel)),std(R.table.Mid_dB(sel)));
    for tau = R.taus
        sel = R.table.Scene == scene & R.table.Tau_ms == tau;
        q = R.table.Primary_dB(sel);
        fprintf('Primary, %3d ms: %.2f +/- %.2f\n',tau,mean(q),std(q));
    end
end
```

```
Centred / independent
Mid baseline: 3.00 +/- 0.03
Primary,  20 ms: 4.33 +/- 0.01
Primary,  80 ms: 7.87 +/- 0.04
Primary, 240 ms: 10.05 +/- 0.10
Panned / independent
Mid baseline: 1.83 +/- 0.03
Primary,  20 ms: 4.33 +/- 0.01
Primary,  80 ms: 7.86 +/- 0.04
Primary, 240 ms: 10.05 +/- 0.09
Centred / correlated
Mid baseline: 0.46 +/- 0.01
Primary,  20 ms: 1.01 +/- 0.02
Primary,  80 ms: 1.25 +/- 0.02
Primary, 240 ms: 1.31 +/- 0.02
```

```
report_figures('study',R);
```

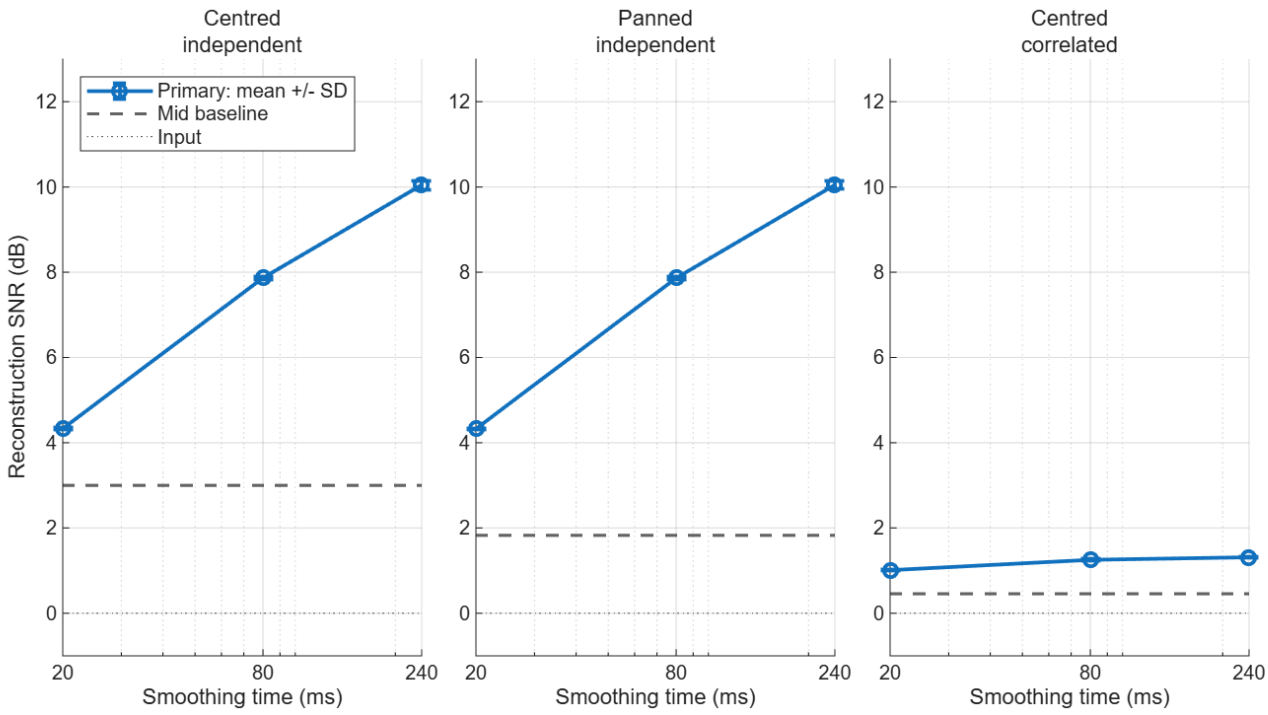


Figure 2. Smoothing-time comparison. Error bars show one sample SD; dashed lines show the mid-channel baseline, and dotted lines the input. Improvements at longer time constants occur in the two independent-noise scenes. The correlated-noise scene remains close to its input quality.

5.2. Time-Frequency Behaviour

Figures 3 and 4 use the panned scene, seed 5306 and tau = 80 ms. This intermediate setting was selected for illustration, not as the best setting. The plotted correlation is the absolute value of Re(P_LR), divided by sqrt(P_LL*P_RR), not complex magnitude coherence. This matches the in-phase synthetic scene.

```
report_figures('maps',R);
```

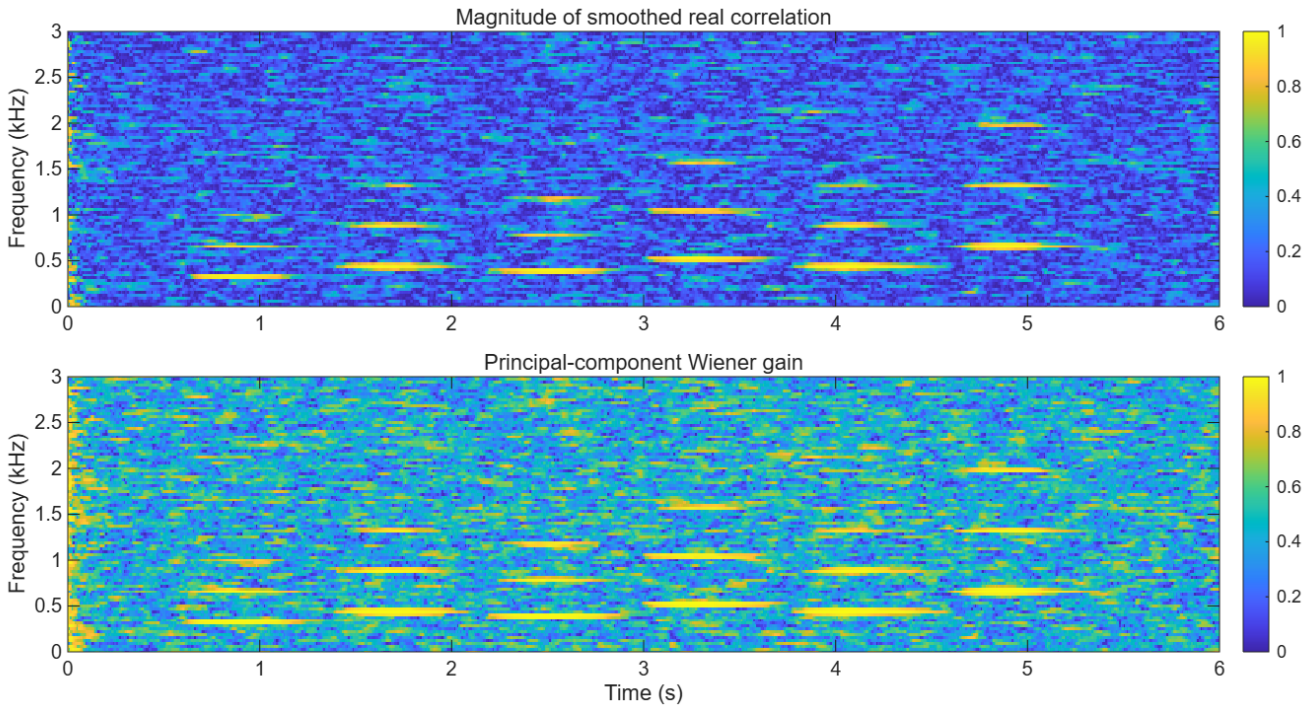


Figure 3. Correlation and Wiener gain (both dimensionless, 0-1). Harmonic tracks retain high gain, while many noise-only bins receive less gain. Random finite-sample covariance can still create nonzero gains in background regions. The method does not use a voice activity detector.

```
report_figures('spectra',R);
```

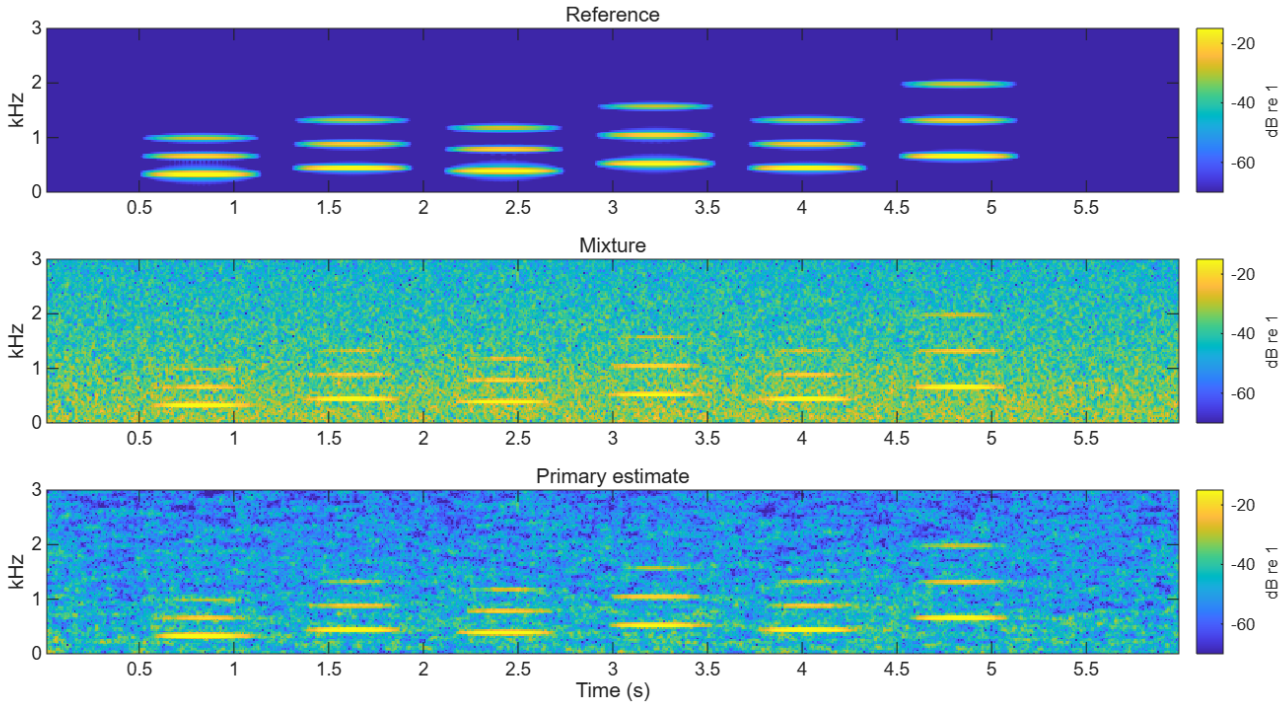


Figure 4. Left-channel STFT magnitudes with a common amplitude scale. The harmonic pattern survives while much of the broadband background decreases. Values are dB relative to unit digital amplitude, not sound pressure level. The full 8 kHz Nyquist range is processed; only 0-3 kHz is displayed to make the notes and their harmonics readable.

5.3. Stereo Image and Audio Examples

Table 2 measures total output ICLD in the panned scene. Target ICLD is 20*log10(0.4) = -7.96 dB; the mid baseline is always 0 dB. This output metric includes residual noise, so it is not a direct estimate of the foreground gain ratio. A value nearer the target is useful but does not establish perceptual localisation accuracy.

```
fprintf('Table 2. Panned-scene output ICLD (dB), mean +/- SD\n');
```

```
Table 2. Panned-scene output ICLD (dB), mean +/- SD
```

```
fprintf('Target: %.2f; mid baseline: 0.00\n',20*log10(0.4));
```

```
Target: -7.96; mid baseline: 0.00
```

```
for tau = R.taus
    sel = R.table.Scene == 2 & R.table.Tau_ms == tau;
    v = R.table.Output_ICLD_dB(sel);
    fprintf('%3d ms: %.2f +/- %.2f\n',tau,mean(v),std(v));
end
```

```
 20 ms: -5.27 +/- 0.08
 80 ms: -6.64 +/- 0.11
240 ms: -7.44 +/- 0.13
```

reference.wav, mixture.wav and primary.wav contain the same example as Figures 3-4. All use one shared export scale. For optional listening, use audioread and sound at the same playback level. No subjective listening observations or intelligibility claims are asserted in this report.

6. Discussion

At 80 ms, mean reconstruction SNR is 7.87 dB for the centred scene and 7.86 dB for the panned scene, compared with mid baselines of 3.00 and 1.83 dB. The approximately 3 dB centred mid-channel gain agrees with averaging two equal-power independent noises. Panning reduces this baseline's score because forcing equal outputs also distorts the target. The adaptive projection retains unequal channel gains; at 240 ms its mean output ICLD is -7.44 dB, compared with the -7.96 dB reference.

Increasing smoothing from 20 to 240 ms raises independent-scene scores from about 4.33 to 10.05 dB. Short smoothing can mistake random channel similarity for a source. Longer averaging reduces that variance here, but a moving source or rapid scene change could invalidate old estimates. No moving-source test was performed, so 240 ms is only the best of the three tested values for these stationary-panning examples.

With rho = 0.8, the 240 ms score is only 1.31 dB. Shared noise contributes a coherent covariance component, violating the isotropic-noise assumption. The estimator cannot reliably distinguish it from the centred music. This is a modelling limitation, not an overlap-add error. Finite samples also produce unequal channel powers despite equal population powers. Synthetic tones, one input SNR and five seeds do not demonstrate general music separation. Real reverberation, phase differences, multiple sources, and perceptual quality require separate evaluation.

7. Conclusion

A self-contained MATLAB experiment demonstrated how stereo covariance and Wiener weighting can recover a coherent musical foreground. The implementation used the Week 8 direct/ambient model with Week 6 frame processing concepts, and validated reconstruction separately from enhancement. Across the tested independent backgrounds, stronger temporal averaging improved reconstruction and brought output level differences closer to the panned reference. Correlated background produced much smaller gains, showing why spatial coherence alone cannot identify a desired source. The objective was met for controlled synthetic examples, while generalisation remains untested. Future work should add changing panning, unequal background powers and recorded material before considering the Week 10 spatial playback applications.

8. References

[1] ELEC5305 teaching materials, "Stereo Decomposition Weights" and "Stereo Decomposition," Week 8A and Week 8B solution Live Scripts, supplied course archive week6-10.zip, 2026.

[2] C. Faller, "Multiple-Loudspeaker Playback of Stereo Signals," Journal of the Audio Engineering Society, vol. 54, no. 11, pp. 1051-1064, 2006. Bibliographic record: AES Journal contents archive, https://www.aes-media.org/historical/pdf/jaes-contents-v50-54.pdf

[3] ELEC5305 teaching materials, "Processing Audio with Overlap and Add: Voice Activity Detection and SNR Measurement," Week 6C Live Script, supplied course archive week6-10.zip, 2026.

[4] MathWorks, "fft" and "ifft," MATLAB documentation, accessed 30 October 2026. https://www.mathworks.com/help/matlab/ref/fft.html and https://www.mathworks.com/help/matlab/ref/ifft.html

9. Reproducibility and AI Use Declaration

The folder contains this Live Script, its exported PDF, editable source, four helper functions, metrics.csv, experiment_results.mat and three WAV examples. Base MATLAB is sufficient; the build was tested in R2026a. Open the Live Script in its folder and select Run All. All numeric results are regenerated from fixed seeds. README.md gives packaging instructions.

OpenAI Codex assisted with experiment design, MATLAB implementation, drafting, debugging and formatting. Numerical results were generated by MATLAB, not invented. The student remains responsible for understanding the analysis and checking the declaration against course requirements.